

Fundamentos de Servicios Web y APIs

Guía visual de conceptos, comunicación HTTP y APIs

Explora los principios esenciales para diseñar, comunicarte e integrar servicios web y APIs modernas.

GET / users
200 OK

```
{  
  :  
}
```

API

¿Qué es un Servicio Web?

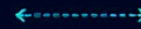
Un servicio web es una aplicación o interfaz que permite la comunicación entre sistemas a través de la web, utilizando estándares abiertos.



Cliente

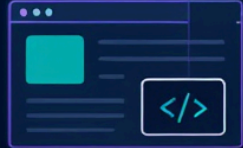


HTTP / Internet



Servicio Web

Comunicación HTTP: el lenguaje de la web



GET

Obtener

Solicita datos de un recurso del servidor.
Ej: GET/usuarios

POST

Crear

Envía datos para crear un nuevo recurso.
Ej: POST/usuarios

PUT

Actualizar

Actualiza completamente un recurso existente.
Ej: PUT/usuarios/1

DELETE

Eliminar

Elimina un recurso del servidor.
Ej: DELETE/usuarios/1

HTTP utiliza **métodos**, **códigos de estado** y formatos como **JSON** para una comunicación clara y efectiva.

Fundamentos de Servicios Web y APIs

Servicios web, APIs y el recorrido de una petición HTTP

01 · Conceptos base

▣ SERVICIO WEB

Investigué que un servicio web es un sistema de software diseñado para soportar la interacción entre máquinas a través de una red. Utiliza protocolos estandarizados como HTTP y formatos de intercambio como XML o JSON, permitiendo que aplicaciones distintas se comuniquen sin importar el lenguaje en el que fueron desarrolladas.

◆ API

API (Interfaz de Programación de Aplicaciones) es un conjunto de reglas, protocolos y herramientas que permiten que dos aplicaciones se comuniquen. Define los métodos y formatos de datos que los programas deben usar para solicitar e intercambiar información de manera estructurada.

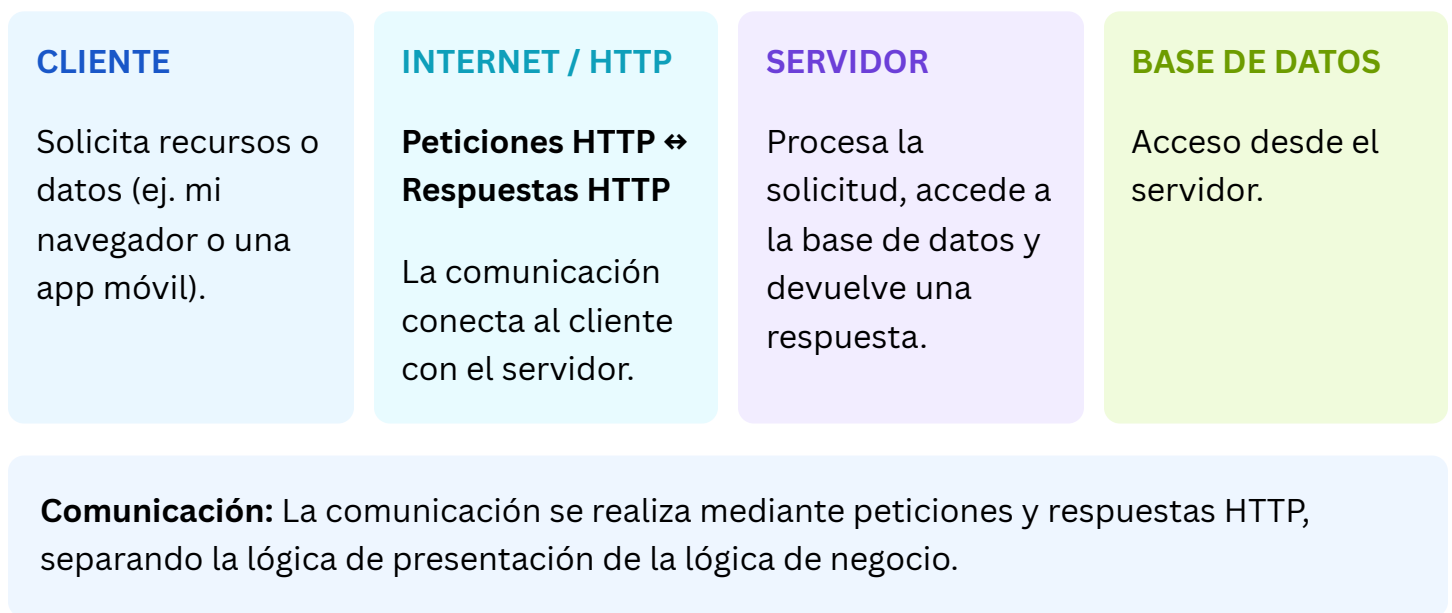
02 · Diferencias entre Servicio Web y API

Analizando ambos conceptos, encontré estas diferencias clave:

Aspecto	Diferencia clave
Alcance	Todo servicio web es una API, pero no toda API es un servicio web.
Ubicación	Las APIs pueden funcionar de forma local (dentro del mismo software), mientras que los servicios web siempre operan a través de una red (internet).
Protocolos	Los servicios web usan protocolos estrictos como HTTP, SOAP o REST.

03 · Arquitectura Cliente-Servidor

Es el modelo fundamental que estudié para la comunicación en la web:



04 · Ciclo de una Petición HTTP

Comprendí que el ciclo funciona en estos pasos:

01 ↓	El cliente resuelve la dirección IP del servidor mediante DNS.
02 ↓	Se establece una conexión TCP/IP.
03 ↓	El cliente envía la petición HTTP con método, URL y encabezados.
04 ↓	El servidor procesa la solicitud y accede a los recursos.
05 ↓	El servidor envía una respuesta con código de estado y datos.
06	El cliente recibe y procesa la respuesta.

05 · Métodos HTTP

Identifiqué los verbos HTTP que indican la acción a realizar:

MÉTODO	ACCIÓN
GET	Obtener o leer recursos existentes (solo lectura).
POST	Crear un nuevo recurso en el servidor.
PUT	Reemplazar completamente un recurso existente.
PATCH	Actualizar parcialmente un recurso existente.
DELETE	Eliminar un recurso del servidor.

06 · Códigos de Estado HTTP más Utilizados

Aprendí que estos códigos indican el resultado de mi petición:

2XX · ÉXITO

200 OK — Petición exitosa.

201 Created — Recurso creado exitosamente.

4XX · ERROR DEL CLIENTE

400 Bad Request — Mi petición tiene sintaxis incorrecta.

401 Unauthorized — Falta autenticación válida.

404 Not Found — El recurso que busco no existe.

5XX · ERROR DEL SERVIDOR

500 Internal Server Error — Error en el servidor.

07 · Formatos de Intercambio de Datos

Comparé los dos formatos principales:

{ } JSON

JSON (JavaScript Object Notation)

Es ligero, fácil de leer para humanos, nativo de JavaScript y es el estándar moderno para APIs REST.

< / > XML

XML (eXtensible Markup Language)

Es más verboso y estricto, usa etiquetas de apertura y cierre, e ideal para documentos complejos.

08 · REST vs SOAP

Diferencié los dos estilos de arquitectura:

REST

Es un estilo arquitectónico ligero y flexible. Usa métodos HTTP nativos y acepta múltiples formatos (JSON, XML). Es el estándar que más se usa hoy en día.

SOAP

Es un protocolo estricto y formal. Solo usa XML, incluye seguridad integrada (WS-Security), pero es más pesado y complejo.

09 · Ejemplos Reales de APIs

Para aterrizar la teoría, investigué tres ejemplos que uso o conozco:

✦ Google Maps API

Permite integrar mapas, geolocalización y rutas en aplicaciones web y móviles.

▣ Stripe API

Es el estándar de la industria para procesar pagos con tarjeta de crédito de forma segura en plataformas web.

✦ OpenAI API

Permite integrar modelos de lenguaje avanzados como ChatGPT en aplicaciones propias para generar texto o automatizar tareas.

IDEA CLAVE · Conclusión Personal

Concluyo que los servicios web y las APIs son como la columna vertebral del software moderno. Mientras **REST y JSON** dominan el desarrollo web actual por su ligereza, comprendo que dominar los fundamentos HTTP y la arquitectura cliente-servidor es esencial para que yo pueda construir aplicaciones escalables y seguras en mi futuro como desarrollador.